

Hybrid Knowledge Graph Embeddings for Ranking Security-Event Technique Links: A Small PyKEEN Feasibility Study

Knowledge Graphs Portfolio

July 27, 2026

Portfolio Cover Pages: Learning Outcomes

The tables below follow the pro forma and map each learning outcome (LO) to the section and page of this report where it is demonstrated. All page references are generated automatically at compile time. For each in-scope LO, tick the proficiency level claimed. Learning outcomes LO3 (Graph Neural Networks) and LO10 (financial KG applications) are intentionally out of scope for this project and are only touched on as possible extensions; no proficiency is claimed for them.

Representations

Learning outcome	Proficiency	Where demonstrated
(LO1) Understand and apply Knowledge Graph Embeddings	☐ basic ☐ exceeded	KGE trained, compared and evaluated: Section 3, page 4
(LO2) Understand and apply logical knowledge in KGs	☐ basic ☐ exceeded	Logic-based representation: Section 4, page 6
(LO3) Understand and apply Graph Neural Networks	<i>out of scope</i>	Not demonstrated; briefly noted as an extension in Section 3.3, page 6
(LO4) Compare different KG data models (database, semantic web, ML, data science)	☐ basic ☐ exceeded	Data model and alternatives: Section 5.2, page 7

Systems

Learning outcome	Proficiency	Where demonstrated
(LO5) Design and implement architectures of a Knowledge Graph	☐ basic ☐ exceeded	Technologies and architecture: Section 2.2, page 4
(LO6) Describe and apply scalable reasoning methods in KGs	☐ basic ☐ exceeded	Reasoning context for the KGE and the rules: Section 3.3, page 6 and Section 4.3, page 7
(LO7) Apply a system to create a Knowledge Graph	☐ basic ☐ exceeded	Construction pipeline: Section 2, page 3
(LO8) Apply a system to evolve a Knowledge Graph	☐ basic ☐ exceeded	KG evolution via KGE and via logic: Section 3.2, page 5 and Section 4.2, page 7

Applications

Learning outcome	Proficiency	Where demonstrated
(LO9) Describe and design real-world applications of KGs	☐ basic ☐ exceeded	Domain and use case: Section 1.1, page 3
(LO10) Describe financial Knowledge Graph applications	<i>out of scope</i>	Not demonstrated; only mentioned as a transfer scenario in Section 1.1, page 3
(LO11) Apply a system to provide services through a KG	☐ basic ☐ exceeded	Service definition and outcome: Section 1.2, page 3 and Section 5.1, page 7
(LO12) Describe the connections between KGs, ML and AI	☐ basic ☐ exceeded	Connections between forms of AI: Section 5.3, page 8

1 Scenario

1.1 Domain (LO9, LO10)

The domain of this knowledge graph application is *enterprise security monitoring*. A modern organisation continuously produces security-relevant events: users authenticate to workstations and servers, processes are started and executed by users, processes belong to recognisable software families, files are accessed, and hosts connect to one another. A Security Operations Centre (SOC) analyst must connect these heterogeneous facts to decide whether a sequence of events corresponds to a known adversary behaviour, such as a MITRE ATT&CK technique. This is a natural knowledge-graph problem because the value lies in the *relationships* between typed entities (users, hosts, processes, files, families, techniques), not in any single event in isolation.

A realistic real-world application built on such a graph would present an analyst with ranked technique hypotheses for a suspicious process, together with the provenance and the neighbouring facts that support each hypothesis. The same architectural idea would also transfer to the financial domain (LO10): banks likewise need to connect infrastructure, users, legal entities, regulations and risks, and KG completion can support hidden-relationship discovery in ownership, compliance or cyber-risk graphs. To be clear, no financial graph is implemented in this project; the construction, embedding and rule layers described below are simply domain-independent, and LO10 remains out of scope.

1.2 Service (LO11)

The service provided by the knowledge graph is a *ranked-query service*: given a process node and the relation `suggests_technique`, return a ranked list of the most plausible ATT&CK-like technique tails. This is the analyst-facing question “given this process, which technique is most plausible?”. Concretely, the project asks whether a compact hybrid knowledge graph—built from synthetic enterprise-security facts plus transparent rule-derived hypotheses—can support link prediction for missing `process` → `suggests_technique` → `technique` triples. The outcome of the project is therefore a small but complete pipeline that constructs the graph, augments it with logic, embeds it, and answers this structured query as a ranked recommendation. The goal is explicitly a feasibility study on a controlled teaching graph, not an operational APT detector, and the results in this report should be read in that light.

2 KG Construction

This part describes how the knowledge graph was constructed (LO7). The code for this section is provided in the ZIP subfolder 2 - `construction`.

2.1 Datasets

The dataset is a deterministic, self-constructed synthetic enterprise-security knowledge graph with 1,232 labelled triples, 245 entities and 9 relations. The observed-style component contains 944 triples representing pseudonymous users, workstations, servers, processes, file kinds and asset roles. A transparent rule layer derives a further 288 facts. The graph labels are semantically grounded in MITRE ATT&CK technique identifiers, but the events and labels are generated locally rather than collected from real incidents: this is a synthetic teaching dataset, not real telemetry.

The dataset is self-constructed and therefore included directly in the ZIP file (folder 2 - `construction`, with `base triples.tsv`, `derived_rule triples.tsv` and `all triples.tsv`); it is also fully reproducible by running the generator. The original project plan was to use the full LANL “Comprehensive, Multi-Source Cyber-Security Events” dataset (<https://lanl.ma.ic.ac.uk/data/cyber1/>), which contains authentication, process, network-flow, DNS and red-team-labelled events. That dataset exceeded 60 GB, and processing at that scale on available hardware (and on Google Colab) led to intractable runtimes and incomplete outputs. An experimental repository documents this effort (<https://github.com/arslansmajevic/knowledge-graphs-project>). As a recovery path, this project uses a small, reproducible, interpretable synthetic dataset that demonstrates the same hybrid architecture and learning-outcome coverage at a tractable scale.

2.2 Technologies and Architecture (LO5)

The system uses Python, with PyKEEN for knowledge-graph embedding and evaluation and a small dependency-free Python module for the symbolic rule layer; data are stored as plain labelled triples in TSV files. The architecture is deliberately modular, mirroring how a production KG system separates raw telemetry, symbolic rules and ML services:

synthetic event facts (observed-style) $\rightarrow$ transparent rules (derived facts) $\rightarrow$ hybrid KG
(triples) $\rightarrow$ PyKEEN KGE (ranking / evaluation).

Each stage is an independent component: `generate_data.py` creates the observed-style triples, `rules.py` derives the symbolic facts with provenance, and `train.py` splits the KG, trains the embedding models and produces ranked output. Keeping ingestion, rule derivation, triple storage, embedding training and ranked output as distinct stages is what makes the pipeline an architecture rather than a single script. In a real deployment, raw telemetry would live outside the embedding model, and only selected, versioned KG facts would be embedded.

2.3 Construction

The graph contains users, workstations, servers, processes, file kinds, process families, security roles and ATT&CK-like technique nodes. Three concrete construction examples illustrate how typed nodes and edges are built from the datasets:

1. **Observed edge.** An observed-style log fact yields an `authenticates_to` edge between a user node and a host node, e.g. `(user_06, authenticates_to, workstation_12)`, materialising the raw telemetry as a typed labelled triple.
2. **Typing edge.** A process is connected to its software family with `has_family`, e.g. `(process_000, has_family, family_powershell)`. This edge later becomes the antecedent that the rule layer reasons over.
3. **Derived edge.** The rule layer creates a `suggests_technique` edge between a process and an ATT&CK-like technique node, e.g. `(process_000, suggests_technique, attack_T1059_command_and_scripting)`, together with a `has_provenance` fact recording that it was produced by the rule rather than observed directly.

Observed relations include `authenticates_to`, `runs`, `executed_by`, `has_family`, `accesses`, `connects_to` and `has_role`; the derived relations are `suggests_technique` and `has_provenance`.

3 ML-based Representation

This part demonstrates Knowledge Graph Embeddings (LO1). The code for this section is provided in the ZIP subfolder 3 - ML.

3.1 The ML Representation and Training (LO1)

The ML-based representation is a knowledge-graph embedding that maps every entity and relation to a continuous vector. Triples were randomly split 80/10/10 into training, validation and test sets using split seed 42. The two models were TransE and ComplEx, trained with PyKEEN using Adam optimisation, learning rate 0.01 and batch size 256. The main metrics are filtered mean reciprocal rank (MRR), Hits@1, Hits@3, Hits@10 and mean rank; the project also reports an application-aligned `technique_tail_type_constrained` metric that ranks only candidate technique tails for `suggests_technique` (and is therefore easier than global link prediction; the two must not be compared numerically).

Five example representations. TransE represents the relation `suggests_technique` as a translation vector, so processes from the same family are moved in a consistent direction toward the same technique node, for example: (1) `process_000` $\rightarrow$ `attack_T1059_command_and_scripting`; (2) `process_009` embedded near `process_000` because they share `family_powershell`; (3) the relation `has_family` as a separate translation linking processes to family nodes; (4) the entity `workstation_12` embedded close to the users that `authenticate_to` it; (5) the technique node `attack_T1003_os_credential_dumping` positioned as the shared tail of all credential-dump-family processes.

True and false positives. A true-positive ranking is (`process_000`, `suggests_technique`, `attack.T1059_command.a`) which the model ranks at the top of the candidate list and which the rule layer also supports. A false-positive ranking is a non-technique entity such as `process_009` or `file_spreadsheet_09` appearing high in the unconstrained candidate list for the same query—these are structurally plausible neighbours but are not valid technique tails, which is exactly why the type-constrained metric is reported separately.

Results. The experiments cover a fair model comparison (Run A), and sensitivity to training duration (Run B), embedding dimension (Run C), a final held-out test (Run D) and seed robustness (Run E). All numbers below are taken directly from the `metrics_comparison.csv` files produced by the pipeline.

Model	MRR	Hits@1	Hits@3	Hits@10	Mean Rank
TransE	0.244	0.110	0.301	0.524	32.00
ComplEx	0.027	0.004	0.008	0.041	100.81

Table 1: Run A. Validation results for TransE and ComplEx, both trained for 80 epochs with dimension 32.

TransE clearly outperformed ComplEx under these conditions. This is a finding about this graph and this training budget, not a universal claim. The synthetic rules create simple translational patterns (same-family processes point to the same technique), which TransE models efficiently; ComplEx is more expressive but that capacity did not help within the same 80-epoch budget.

Epochs	MRR	Hits@1	Hits@3	Hits@10	Mean Rank
20	0.159	0.041	0.195	0.390	52.14
80	0.244	0.110	0.301	0.524	32.00
200	0.361	0.232	0.419	0.602	20.76

Table 2: Run B. Validation results for TransE (dimension 32) at different epoch counts.

Validation improved from 20 to 80 to 200 epochs, indicating the 80-epoch model had not fully converged. If the final model is selected strictly on validation, the 200-epoch configuration is the most consistent next choice.

Dimension	MRR	Hits@1	Hits@3	Hits@10	Mean Rank
16	0.299	0.179	0.333	0.549	26.16
32	0.244	0.110	0.301	0.524	32.00
64	0.213	0.057	0.305	0.528	32.31

Table 3: Run C. Validation results for TransE (80 epochs) at different embedding dimensions.

The 16-dimensional model had the best global validation MRR, even though the 64-dimensional model had the highest type-constrained technique-tail MRR. The graph is small and regular, so a compact embedding suffices; more capacity is not automatically better, and global and task-specific metrics must be interpreted separately.

The test MRR of 0.240 is close to the validation MRR of 0.244 for the same configuration, so the validation estimate was not wildly optimistic. Hits@10 of 0.532 means the true held-out triple was ranked in the top 10 in just over half of the evaluated cases—meaningful for a feasibility study, but insufficient for an operational system.

The three test MRR values (0.256, 0.240, 0.296) have a mean of about 0.264 and a standard deviation of about 0.029, indicating moderate stability across seeds.

3.2 Evolving the KG with the ML Representation (LO8)

The ML representation is used to *complete* the knowledge graph. Link prediction proposes missing `suggests_technique` edges: a held-out true triple is the KG-completion task, and a top-ranked candidate is a new hypothesis edge that could be added to the graph. The effect on evolution is therefore *adding*

Model	MRR	Hits@1	Hits@3	Hits@10	Mean Rank
TransE	0.240	0.101	0.286	0.532	30.74

Table 4: Run D. Final test result for the available TransE 80-epoch, 32-dimensional configuration.

Seed	MRR	Hits@1	Hits@3	Hits@10	Mean Rank
7	0.256	0.097	0.347	0.577	25.10
42	0.240	0.101	0.286	0.532	30.74
1337	0.296	0.177	0.327	0.556	29.27

Table 5: Run E. Test robustness across three model seeds for TransE (80 epochs, dimension 32).

a candidate edge. Because KGE scores are rankings rather than calibrated probabilities, each proposed edge is treated as a candidate update for analyst review, not as confirmed truth.

3.3 Context and Limitations (LO6)

The KGE performs *approximate* reasoning: it ranks plausible missing links rather than deriving them deterministically, which is the scalable completion counterpart to the exact rule reasoning of Section 4.3. On scaling: *volume*—PyKEEN’s filtered evaluation and negative sampling grow with the number of entities and triples, so a 60 GB log graph would need batching, sampling and likely GPU training. *Variety*—adding new relation types is cheap for the triple model but dilutes the regular translational pattern that made TransE effective here. *Veracity*—with little or noisy training data, embedding quality degrades quickly, as the seed-robustness spread already hints. In our concrete scenario, the model only ever sees four technique families, so its apparent strength would not transfer to the full ATT&CK matrix without far more, and more diverse, training evidence. Two further honest caveats: the random triple split is appropriate for a teaching benchmark but could leak temporal information in a real log setting (a real extension needs a chronological split), and the ranking metrics test recovery of withheld triples, not alert precision or recall on independently labelled incidents. Graph Neural Networks (LO3) were not applied in this project; a relational GNN over the same graph would be a natural extension, but no claim is made about it here.

4 Logic-based Representation

This part demonstrates logical knowledge in KGs (LO2). The code for this section is provided in the ZIP subfolder 4 - logic.

4.1 The Logic Representation (LO2)

The logic-based representation is a small, inspectable, deterministic Datalog-style rule set, kept deliberately separate from model training. The core rule is:

```
has_family(Process, Family) and family_maps_to_technique(Family, Technique) imply suggests_technique(Process, Technique).
```

Five concrete examples of the logic-based representation:

1. `family_powershell` $\mapsto$ `attack_T1059_command_and_scripting`.
2. `family_remote_desktop` $\mapsto$ `attack_T1021_remote_services`.
3. `family_downloader` $\mapsto$ `attack_T1105_ingress_tool_transfer`.
4. `family_credential_dump` $\mapsto$ `attack_T1003_os_credential_dumping`.
5. For every derived `suggests_technique` fact the rule also *creates* a new `has_provenance` edge (`rule_family_to_attack_technique`), so the symbolic reason for a suggestion is stored explicitly in the graph.

Example 5 illustrates a rule that *creates* new nodes/edges in the graph. To be explicit about scope: the implemented rule set is a single non-recursive step and does not implement full recursive or existential Datalog reasoning. A natural recursive extension is described in Section 4.3 (following `connects_to` chains for lateral-movement detection across arbitrarily many edges).

4.2 Evolving the KG with the Logic Representation (LO8)

The logic-based representation is used to *complete* the knowledge graph by *adding* new facts: each firing of the rule adds a `suggests_technique` edge and a `has_provenance` edge that were not present in the observed-style data. Unlike the KGE, this addition is deterministic and fully auditable—given the same observed facts, the same derived facts are produced—which is why the derived triples are stored with provenance and used as transparent ground for the embedding layer.

4.3 Context and Limitations (LO6)

The rule engine performs *deterministic, exact* reasoning, which is the complement to the KGE’s approximate completion. On scaling within our scenario: *volume*—deterministic forward chaining over a single non-recursive rule is linear in the number of `has_family` facts and scales well, but a recursive lateral-movement rule over `connects_to` chains could grow combinatorially and would need stratification or bounded recursion. *Variety*—the current rule covers only four families, so extending it to the full ATT&CK matrix means authoring and maintaining many more mappings. *Veracity*—the rule trusts its `has_family` input, so noisy or spoofed family labels would propagate directly into wrong technique suggestions. A concrete improvement is to add a recursive rule that follows host-to-host `connects_to` edges, plus richer provenance and temporal conditions, so the logic layer can express multi-hop attack chains rather than a single family-to-technique step.

5 Reflection

The supporting material for this part is provided in the ZIP subfolder 5 - `reflection`.

5.1 Outcome of the Service (LO11)

The service formulated in Section 1.2 was delivered: the pipeline answers “given this process, which technique is most plausible?” as a ranked list, and the final TransE test run places the true held-out technique in the top 10 for just over half of the evaluated cases (Hits@10 = 0.532). As a ranked-query service this is a working, if modest, outcome: it demonstrates feasibility on a controlled graph. It is not yet a deployable analyst tool because the scores are rankings, not calibrated risk values, and the technique labels are rule-derived rather than independently observed. Human review therefore remains necessary. The most consistent improvement to the service is to run the final test for the strongest validation configuration (TransE, 200 epochs, dimension 32) and to surface provenance and neighbouring facts alongside each ranked hypothesis.

5.2 Data Model and Alternatives (LO4)

The chosen data model is a labelled-triple representation. The generator naturally resembles a *property graph* from the database community, because security objects are typed entities connected by labelled edges; PyKEEN then consumes the same content as labelled triples, which is close to the *RDF-style* statement view from the semantic-web community. Compared with a pure relational model, the triple/property-graph view makes multi-hop relationship queries (the core of the scenario) far more natural, at the cost of weaker native support for tabular aggregation. Compared with a feature-matrix view from the machine-learning and data-science communities, the graph view preserves explicit relations and provenance that a flattened feature table would discard. The triple model was chosen because the project’s value lies in relationships, and because it is the shared substrate that both the symbolic rules and the embedding model can operate on without conversion. This is a modelling choice suited to this project, not a claim that one data model is universally better.

5.3 Connections between Forms of AI (LO12)

The project combines three forms of knowledge: symbolic rules, graph data modelling and neural embeddings. The interaction in this scenario is one-way and sequential: the logic layer produces transparent `suggests_technique` and `has_provenance` facts, and the embedding layer then generalises over the whole graph, including those facts, to rank missing links. This interaction could be improved by making it bidirectional—for example, using rule provenance to mask rule-derived edges during evaluation so the KGE is measured on genuinely new generalisation, or using high-confidence KGE predictions to propose candidate new rules. It would be partly possible to learn the logical mappings automatically (e.g. rule mining over observed family/technique co-occurrences), and the KGE’s accuracy could in turn be improved by logical constraints that forbid type-invalid tails (the false positives noted in Section 3.1). Because the targets are currently rule-derived, the embedding partly relearns the rule; the clearest path to a genuinely hybrid system is to introduce independently sourced labels so that the logic and ML layers contribute complementary, rather than overlapping, knowledge. Neither component alone makes the system a complete AI-based security product.